

Financial Statement Review Agent

Project 2 — design of the solution described in the BRD

Field	Value
Document name	High-Level Design (HLD) including data diagrams and UI screens
Document ID	FSRA-HLD-001
Input document	FSRA-BRD-001 v1.1 (frozen)
Version	1.1
Date	8 September 2026
Status	Draft — platform-ready

1. Purpose of this document

The BRD says **what** the service must do. This design says **how it is structured** and **how it looks**.
Build starts only after this design is reviewed against BR-01 to BR-11.

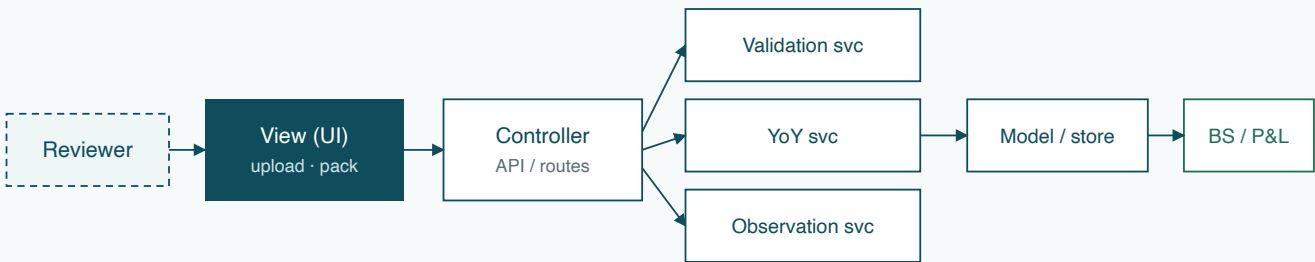
2. Design rules (from the BRD)

Rule	Design meaning
Numbers are checked by rules	Math, consistency, and YoY engines produce pass/fail and amounts. GenAI does not invent totals.
Show evidence	Every finding and observation carries line name, year, amount, and check id.
Explain, do not invent	Observation text is written only from rule results + statement lines already retrieved.
Clear review pack	UI shows summary, findings, variances, and observations in one place.

3. High-level architecture

MVC + Service layers. Names are roles, not product brands.

Figure 1 — system architecture



View → Controller → Services → Model. Validation and YoY own numbers. Observation svc explains rule output only.

3.1 What each block does

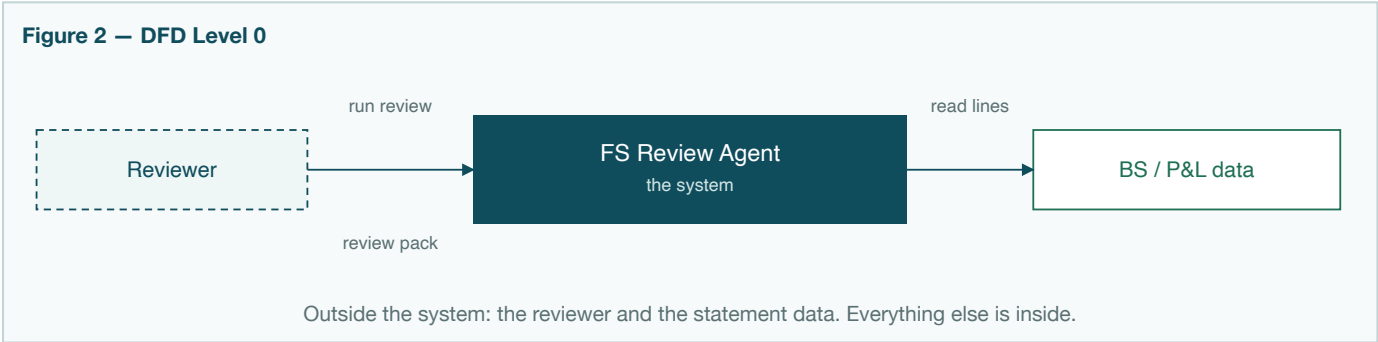
Block	Responsibility
View (UI)	Upload / select company years, run review, show pack. Never invent amounts on screen.
Controller	Accepts review request, calls services in order, returns one review pack DTO.
Validation service	Math totals + Balance Sheet identity (+ other simple consistency rules).
YoY service	Compares current vs prior; flags material absolute / % change.
Observation service	Turns findings + variances into short review comments with evidence attached.
Model / store	Normalized statement lines, review runs, findings, observations.

3.2 Architecture alternatives (why this one)

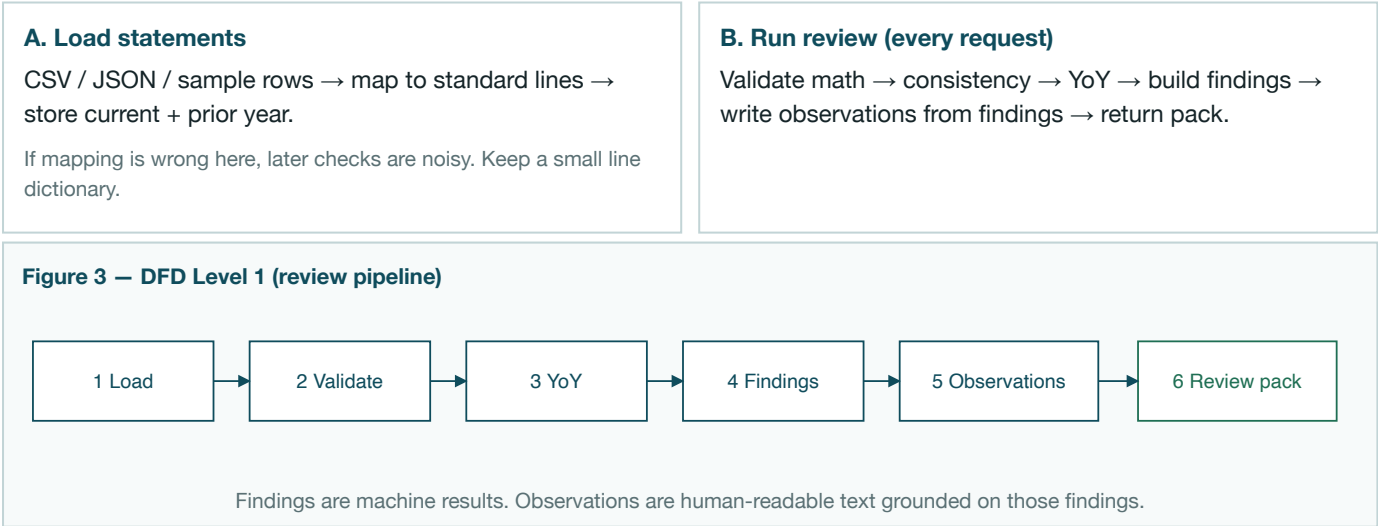
Option	Verdict
A. Pure LLM reads the spreadsheet and “reviews”	Rejected — numbers can be hallucinated; weak evidence.
B. Rules only, no GenAI comments	Possible but weaker UX for “review observations”.
C. Rules for truth + GenAI for wording (chosen)	Selected — matches BR-05/BR-06; auditable; demo-ready.

4. Data flow diagrams

4.1 Context (Level 0)



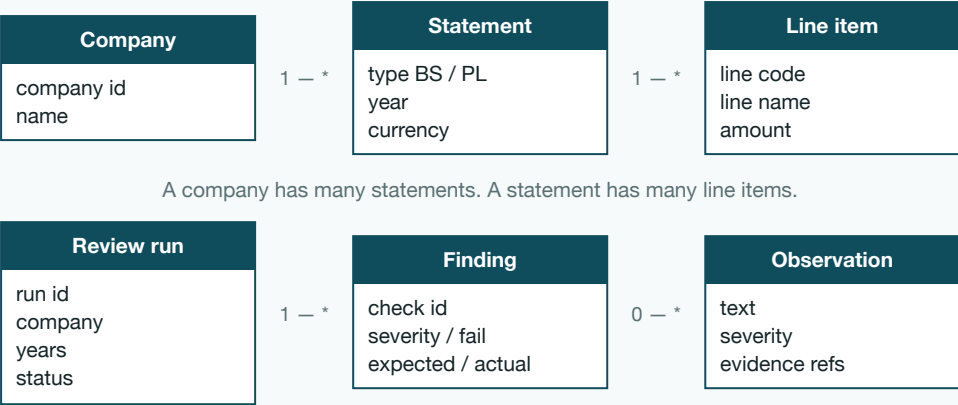
4.2 Review pipeline (Level 1)



5. Data model

What must be stored so evidence and re-runs are possible.

Figure 4 — entity relationship (logical)



A run produces findings. Observations point back to findings / line items — never free-floating numbers.

6. Sequence — one review run

Figure 5 — happy path and fail path

#	From → To	What happens
1	Reviewer → UI	Selects company / years (or uploads sample CSV).
2	UI → Controller	POST review with company_id + years.
3	Controller → Model	Load normalized BS / P&L lines for both years.
4	Controller → Validation	Run math + consistency checks → findings.
5	Controller → YoY	Compute variances above threshold → variance findings.
6	Controller → Observation	Write short comments using only findings + line evidence.
7	Controller → UI	Return review pack: summary, findings, variances, observations.

BRD scenario	Which path
Totals do not add	Step 4 fail finding → step 6 observation citing expected vs actual.
Revenue +40% YoY	Step 5 material variance → step 6 explanation with both years.
Assets ≠ L+E	Step 4 consistency fail → highlighted in summary.

7. UI design (screens)

Three frames are enough for the demo. Keep one navy, one fail rust. No extra dashboards in v1.

Frame A — Select / upload	Frame B — Findings	Frame C — Observations
FS Review Agent Assistive review · Not a statutory audit opinion. Company: Acme Demo Co Years: 2023 (current) · 2022 (prior) Statements: Balance Sheet + P&L Run review →	FS Review Agent Assistive review · Not a statutory audit opinion. MATH-01 Total assets · Pass CONS-01 Assets ≠ L+E · Fail expected 0 · actual 1.2m YOY-12 Revenue · +40% 2022 100 · 2023 140 Summary · Findings · YoY	FS Review Agent Assistive review · Not a statutory audit opinion. BS does not balance. Difference 1.2m on CONS-01. Revenue rose 40% YoY (100 → 140). Review drivers. Evidence chips link each comment to lines / checks. Export pack

8. Low-level design (modules) + environments

Module	Does	Must not
View	Upload, run, show pack / export.	Compute secret totals only in the browser.
Controller	Orchestrate services; return DTO.	Embed LLM prompts with raw unvalidated numbers only.
Validation svc	Math + consistency rules.	Skip evidence fields on fail.
YoY svc	Thresholded variance list.	Hide prior-year value.
Observation svc	GenAI wording from findings.	Invent amounts not in findings.
Notify / Health	Mail/webhook on complete, critical, down.	Spam; alert without run/trace id.
Model	Persist lines, runs, findings, feedback.	Overwrite source amounts silently.

8.1 Environments

Env	Purpose
local	Developer machine; sample CSV; optional LLM key.
dev / test	Automated tests for rules; known good / known bad fixtures.
demo	Deployed UI + API for hackathon presentation.

8.2 Review pack returned to the UI (logical fields)

Field	Meaning
run id	Which review this pack belongs to.
summary	Counts: passed, failed, material variances.
findings[]	check id, status, expected, actual, lines[].
variances[]	line, prior, current, abs, pct, material?
observations[]	text, severity, evidence refs.
trace id	Correlation id for logs, alerts, and feedback.

9. Think big — platform capabilities (still simple)

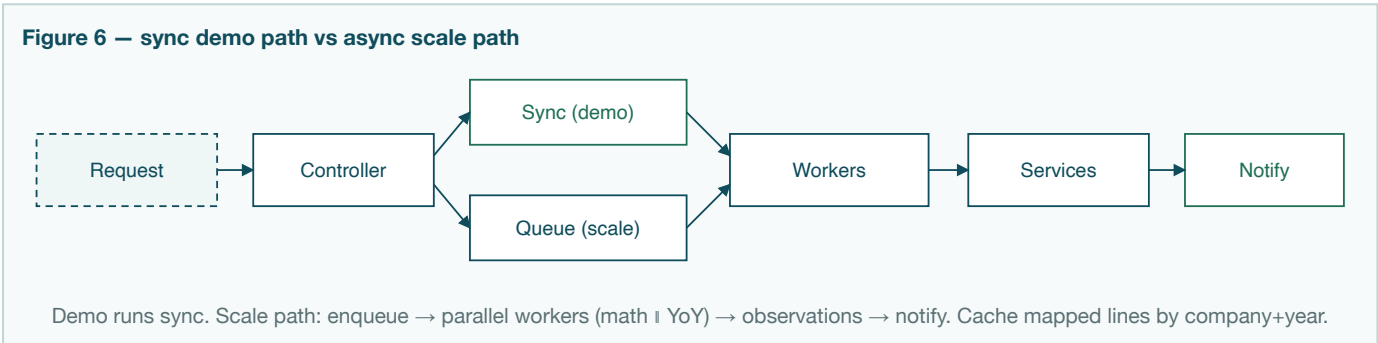
Core review stays the same. These blocks make the agent enterprise-ready and score on integration, performance, monitoring, and reuse.

9.1 Integration alternatives (plug into any environment)

Channel	How it connects	When to use
REST API	POST /reviews, GET /reviews/{id}, GET /health	Any backend, ERP adapter, RPA bot
Webhook out	Push review-complete / critical-fail JSON to a URL	Ticketing, Slack/Teams bots, workflow tools
UI embed	Same API behind Streamlit / portal iframe	Finance team day-to-day
File drop	Watch folder / S3 → enqueue review	Batch month-end packs
Chat surface	Teams/Slack command calls the same Controller	“Review Acme 2023” from chat

One Controller contract. Many fronts. That is the reusable design.

9.2 Pipeline + traffic (optimization)



Lever	Design
Pipelining	Validation and YoY are independent → run in parallel workers; Observation waits for both.
Traffic	API rate limit + job queue; return 202 Accepted + run id under load.
Cache	Normalized line map cached; invalidate on re-upload.
Idempotency	Same company+years+hash → reuse pack (no double GenAI spend).

9.3 Notifications, health, auto-mail

Event	Who gets it	Channel
Review completed	Requesting reviewer	In-app + email / webhook
Critical finding (MATH/CONS fail)	Reviewer + optional finance lead	Email subject: company · check id · run id
Service unhealthy	Ops / on-call	Health probe fails → auto email (and optional Slack)
Job stuck / timeout	Ops	Alert with trace id

Health contract: GET /health checks API process, DB/store reachability, and (optional) GenAI key configured. Down → alert. No silent failure.

9.4 Trace + feedback loop

Trace (every run) • trace / run id on every log line • who started, when, company, years • counts: pass / fail / material • duration of each service step	Feedback (every observation) • Reviewer marks useful / not useful • Stored against observation + check id • Feeds prompt / template tuning • Shown in demo as continuous improvement
---	--

9.5 Monitoring (what we watch)

Metric	Why judges care
p95 review latency	Performance
Queue depth / worker count	Traffic handling
Rule fail rate by check id	Model / rule quality
Health uptime + alert count	Process of monitoring
Feedback useful %	Observation quality over time

10. Traceability to the BRD

BRD	Architecture	Data	UI
BR-01 BS + P&L years	Load in Controller / Model	Company, Statement, Line	Frame A
BR-02 math accuracy	Validation service	Finding (MATH-*)	Frame B pass/fail
BR-03 YoY	YoY service	Finding / variance list	Frame B YoY rows
BR-04 consistency	Validation CONS-*	Finding with expected/actual	Fail row + chip
BR-05 observations	Observation service	Observation + evidence	Frame C
BR-06 no invented numbers	Observation reads findings only	Evidence refs required	Chips show source amounts
BR-07 clear summary	Review pack DTO	Run summary counts	Summary strip
BR-08 notify	Notify service + webhook	Notification log	Toast / mail demo
BR-09 API anywhere	REST + webhook channels	Same pack DTO	UI is one client only
BR-10 down → mail	Health + alerter	Alert events	Ops mail template
BR-11 trace + feedback	Trace middleware + feedback API	Run audit + votes	👍 / 👎 on observation

11. Design review (before build)

#	Check against the frozen BRD
1	Figure 1 separates numeric services from GenAI observation service.
2	Figure 4 stores findings with expected/actual; observations reference them.
3	Figure 5 covers math fail, YoY material, and consistency fail.
4	Frames A–C exist; evidence chips appear on fails and observations.
5	Environments and MVC/service split are named for implementation.
6	Section 9 covers integrate / pipeline / notify / health / trace / feedback.
7	API-first: UI is a client, not the only door into the agent.